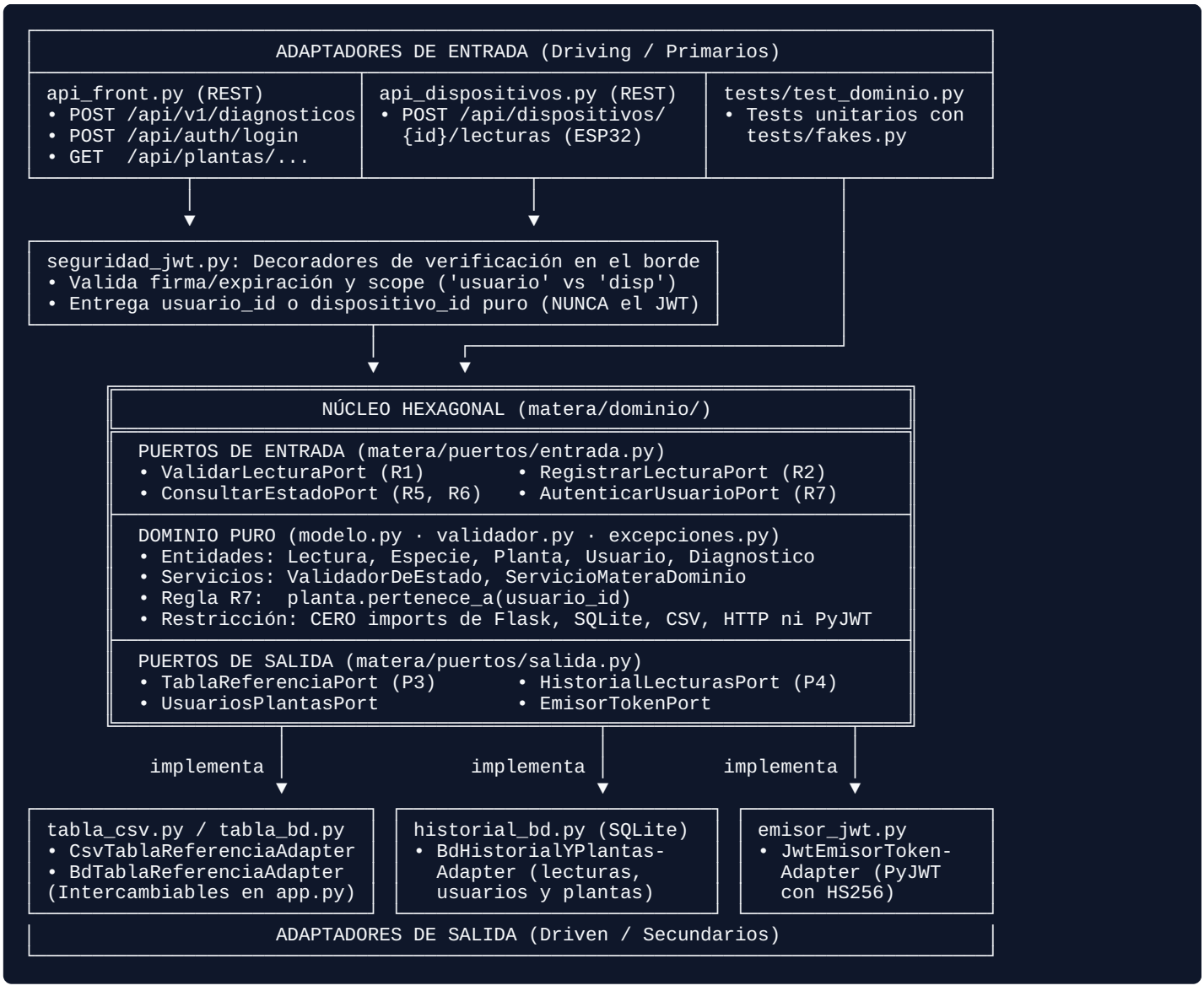
Documento de Arquitectura — Fase 2: Del Sensor al Dominio

Proyecto Matera Inteligente · SmartPlantBase
Asignatura: Arquitectura de Software · Universidad Sergio Arboleda · Semestre 2026-3
Autores: Esteban Castro y equipo · **Framework y Lenguaje:** Python 3.14 / Flask

1. Diagrama Hexagonal del Sistema

La arquitectura implementa el patrón **Puertos y Adaptadores (Alistair Cockburn)**, aislando las reglas botánicas y de propiedad en un núcleo de dominio desacoplado de frameworks, protocolos e infraestructura física:



2. Tabla de Puertos y sus Adaptadores

Puerto	Tipo	Archivo del Puerto (Interfaz)	Archivo del Adaptador (Implementación)	Responsabilidad
`ValidarLecturaPort`	Entrada	`matera/puertos/entrada.py`	`matera/dominio/validador.py` *(consumido por `api_front.py`)*	Evalúa una lectura manual contra la especie (R1) y lista especies de referencia.
`RegistrarLecturaPort`	Entrada	`matera/puertos/entrada.py`	`matera/dominio/validador.py` *(consumido por `api_dispositivos.py`)*	Recibe telemetría de un sensor ESP32 (R2), evalúa salud botánica y persiste en el historial (R5).
`ConsultarEstadoPort`	Entrada	`matera/puertos/entrada.py`	`matera/dominio/validador.py` *(consumido por `api_front.py`)*	Consulta última lectura agregada e historial por planta verificando pertenencia (R5, R6, R7).
`AutenticarUsuarioPort`	Entrada	`matera/puertos/entrada.py`	`matera/dominio/validador.py` *(consumido por `api_front.py`)*	Registra usuarios, valida credenciales y solicita emisión de tokens de sesión (R7).
`TablaReferenciaPort`	Salida	`matera/puertos/salida.py`	`matera/adaptadores/salida/tabla_csv.py` `matera/adaptadores/salida/tabla_bd.py`	Obtiene rangos óptimos de especies. Conmutable por configuración (`CSV` vs `SQLite`) (R4).
`HistorialLecturasPort`	Salida	`matera/puertos/salida.py`	`matera/adaptadores/salida/historial_bd.py`	Persiste lecturas y diagnósticos históricos y consulta la lectura más reciente en SQLite (R5).
`UsuariosPlantasPort`	Salida	`matera/puertos/salida.py`	`matera/adaptadores/salida/historial_bd.py`	Persiste y consulta usuarios, plantas registradas y relación dispositivo-planta (R7).
`EmisorTokenPort`	Salida	`matera/puertos/salida.py`	`matera/adaptadores/salida/emisor_jwt.py`	Firma tokens criptográficos JWT segregados para usuario y dispositivo usando PyJWT (R7).

3. Manejo de Tokens JWT y Regla de Propiedad (R7)

El manejo de seguridad se implementa de manera perimetral y desacoplada:

1. **Emisión del Token (`Puerto de Salida: EmisorTokenPort`):**
   - El caso de uso en el dominio (`AutenticarUsuarioPort`) valida las credenciales y solicita al puerto de salida `EmisorTokenPort.emitir_token_usuario(usuario_id, email)` o `emitir_token_dispositivo(dispositivo_id, planta_id)` la generación del token.
   - La clase `JwtEmisorTokenAdapter` en `matera/adaptadores/salida/emisor_jwt.py` es el **único lugar del sistema que importa `PyJWT`** y firma los claims (`sub`, `scope`, `iat`, `exp`) usando el algoritmo HS256.
2. **Verificación del Token (`Adaptador de Entrada: seguridad_jwt.py`):**
   - La petición HTTP incluye la cabecera `Authorization: Bearer <token>`.
   - Los decoradores `@requiere_jwt_usuario` y `@requiere_jwt_dispositivo` verifican firma, expiración y segregan el acceso según el claim `scope`:
     - Un token con `scope="dispositivo"` que intenta acceder a `/api/plantas` es rechazado con **`403 Forbidden`**.
     - Un token emitido para `esp32-01` que intenta enviar telemetría a `/api/dispositivos/esp32-02/lecturas` es rechazado con **`403 Forbidden`**.
   - El decorador extrae el identificador verificado (`usuario_id: str` o `dispositivo_id: str`) y lo inyecta como parámetro plano al puerto de entrada.
3. **Por qué el Dominio NUNCA ve el Token:**
   - Si el dominio recibiera strings JWT, violaría la inversión de dependencias y quedaría acoplado al estándar RFC 7519 y a `PyJWT`. El dominio sólo procesa conceptos del negocio: *"¿pertenece esta planta al usuario autenticado?"*.
4. **La Regla de Propiedad vive en el Dominio (`matera/dominio/validador.py`):**

```
# matera/dominio/validador.py (ConsultarEstadoPort)
planta = self._usuarios_plantas.obtener_planta_por_id(planta_id)
if not planta.pertenece_a(usuario_id):
    raise AccesoNoAutorizadoPlantaError(usuario_id, planta_id)
```

Si un usuario legítimo (Carlos, `usr-carlos`) intenta consultar la planta de otro usuario (Ana, `planta-01`), el dominio lanza `AccesoNoAutorizadoPlantaError`, y el manejador HTTP centralizado responde **`403 Forbidden`** (**`ACCESO_DENEGADO`**).

---

4. Justificación de Principios SOLID

- **Principio de Inversión de Dependencias (DIP):** El dominio define la interfaz abstracta `TablaReferenciaPort(ABC)` con el método `obtener_por_nombre(nombre: str) -> Optional[Especie]`. Tanto `CsvTablaReferenciaAdapter` como `BdTablaReferenciaAdapter` dependen de esa abstracción. El evaluador botánico jamás importa `sqlite3` ni `csv`.
- **Principio Abierto/Cerrado (OCP):** Conmutar entre CSV y SQLite (R4) o conectar el simulador ESP32 (R2) se logró creando nuevos adaptadores sin tocar una sola línea de `matera/dominio/modelo.py` ni `matera/dominio/validador.py`. Del mismo modo, una nueva variable ambiental se incorpora en `LIMITES_FISICOS` sin alterar el evaluador.
- **Principio de Segregación de Interfaces (ISP):** Los puertos de salida están segregados por cliente y propósito: `TablaReferenciaPort` solo expone consulta de especies; `HistorialLecturasPort` solo gestiona telemetría; `EmisorTokenPort` solo firma tokens. El adaptador de dispositivos (`api_dispositivos.py`) solo conoce `RegistrarLecturaPort`.
- **Principio de Responsabilidad Única (SRP):** Cada clase tiene un único motivo de cambio: `ValidadorDeEstado` cambia por reglas botánicas; `seguridad_jwt.py` por políticas de autenticación HTTP; `historial_bd.py` por cambios en el esquema SQL.

---

5. Prueba de Cambio (15% de la Rúbrica)

- **Commit de la Prueba de Cambio:** `8c4b6d8685aeb2bf2784258d349550a6cbf25812`
- **Mensaje del Commit:** `feat(dispositivos): adaptador de dispositivos y simulador ESP32 (Prueba de Cambio)`
- **Archivos creados / modificados en el commit:**

```
+ matera/adaptadores/entrada/api_dispositivos.py
+ simulador_esp32.py
+ tests/test_api_dispositivos.py
~ matera/app.py (únicamente registro del blueprint api_dispositivos)
```

- **Auditoría de Invarianza del Dominio:** Al ejecutar `git diff 8c4b6d8~1 8c4b6d8 -- matera/dominio/ matera/puertos/`: **El resultado es estrictamente vacío: EXACTAMENTE CERO (0) ARCHIVOS MODIFICADOS.**
  Esto demuestra que la incorporación de un nuevo dispositivo físico sensor (R2, R3) no requirió ninguna adaptación en las reglas de negocio, satisfaciendo al 100% la restricción arquitectónica del enunciado.

---

6. Preguntas de Cambio: Migración a MQTT

Si mañana las lecturas ambientales llegaran mediante mensajes de un broker MQTT (ej. topic `matera/dispositivos/{id}/telemetria`) en lugar de solicitudes HTTP:

1. **Archivos que se crearían (Nuevos adaptadores de entrada):**
   - `matera/adaptadores/entrada/suscriptor_mqtt.py`: Cliente MQTT (ej. `paho-mqtt`) que se suscribe a los topics de telemetría, deserializa el payload JSON recibido, valida el token de dispositivo si aplica y llama directamente a `RegistrarLecturaPort.registrar(dispositivo_id, valores, marca_tiempo)`.
   - `tests/test_suscriptor_mqtt.py`: Pruebas de integración para verificar la recepción y despacho de mensajes MQTT.
2. **Archivos existentes que se modificarían (Únicamente Composición):**
   - `matera/app.py`: Se instanciaría el cliente MQTT pasando el mismo `servicio_dominio` existente (que implementa `RegistrarLecturaPort`) e iniciando el bucle de escucha en segundo plano (`client.loop_start()`).
   - `requirements.txt`: Se agregaría la librería de cliente MQTT (`paho-mqtt`).
3. **Archivos que NO cambiarían y por qué:**
   - **`matera/dominio/*`** (**`modelo.py`**, **`validador.py`**, **`excepciones.py`**)**:** No cambian porque el núcleo botánico desconoce el protocolo de transporte (HTTP, MQTT, WebSockets o consola).
   - **`matera/puertos/*`** (**`entrada.py`**, **`salida.py`**)**:** No cambian porque `RegistrarLecturaPort` ya declara la firma agnóstica exacta requerida: `registrar(dispositivo_id: str, valores: Mapping, marca_tiempo: str)`.
   - **`matera/adaptadores/salida/*`** (**`tabla_csv.py`**, **`tabla_bd.py`**, **`historial_bd.py`**)**:** No cambian porque el mecanismo para almacenar la telemetría recibida en la base de datos SQLite y consultar los rangos de referencia sigue siendo idéntico.